TASK 1 – Understand the Problem and Identify the Data

1.1 Problem Analysis

The Ksheera Village Milk Producers' Co-operative collects milk from approximately 300 farmers twice a day. The payment depends on the quantity of milk, fat percentage and solids-not-fat (SNF) percentage.

The standard rate is ₹30.00 per litre for milk containing 3.5% fat and 8.5% SNF.

- For every 0.1% increase or decrease in fat, the rate changes by ₹0.40.
- For every 0.1% increase or decrease in SNF, the rate changes by ₹0.30.
- Payments are calculated every ten days.

The program must calculate each farmer's payment accurately and determine the total amount payable for ten days.

1.2 Inputs and Outputs

Data	Type	Explanation
Farmer ID	String (str)	Identifies each farmer uniquely.
Milk quantity	Float or Decimal	Stores the quantity of milk in litres.
Fat percentage	Float or Decimal	Stores the fat content of milk.
SNF percentage	Float or Decimal	Stores the solids-not-fat content.
Shift	String (str)	Identifies the morning or evening shift.
Base rate	Numeric constant	Standard rate of ₹30.00 per litre.
Fat adjustment	Numeric constant	₹0.40 for every 0.1% fat step.
SNF adjustment	Numeric constant	₹0.30 for every 0.1% SNF step.
Rate per litre	Decimal	Calculated price per litre.
Payment amount	Decimal	Total payment for a farmer's entry.
Ten-day total	Decimal	Total amount payable over ten days.

Justification: Farmer IDs and shifts are text because they identify entries rather than represent quantities. Milk quantity and percentages may contain fractional values. Money must be represented accurately to avoid discrepancies in bank payments.

For the final implementation, Decimal is recommended for monetary calculations.

1.3 Named Constants

BASE_RATE = 30.00

STANDARD_FAT = 3.5

STANDARD_SNF = 8.5

FAT_STEP_VALUE = 0.40

SNF_STEP_VALUE = 0.30

STEP_SIZE = 0.1

MIN_FAT = 2.0

MAX_FAT = 7.0

These values are stored as named constants because they remain unchanged during a calculation run.

If the co-operative revises the rate, the programmer can update the constants in one place instead of searching for repeated numerical values.

1.4 Validation Rules

The program must satisfy the following rules:

1. Farmer ID must not be empty.
2. Milk quantity must be greater than zero.
3. Fat percentage must be between 2.0% and 7.0%, inclusive.

4. SNF percentage must be a valid, non-negative number.
5. The shift must be either morning or evening.
6. All required inputs must be valid before calculating payment.
7. The rate per litre must be rounded to two decimal places.
8. Each payment must be rounded to two decimal places.
9. The ten-day total must equal the sum of the individual payments.
10. Invalid entries must be rejected with an appropriate error message.

The permitted SNF range is not specified in the question. Therefore, the program should use a validated range provided by the co-operative rather than inventing an official limit.

1.5 Assumptions

- Every farmer has a unique ID.
- Milk quantity can contain fractional litres.
- Fat and SNF values are recorded as percentages.
- The same rate card applies throughout a calculation period unless a seasonal revision is introduced.
- Payments are made in Indian rupees.
- Each individual payment is rounded to the nearest paisa.
- Fat and SNF values are assumed to be valid analyser readings after input validation.

1.6 Alternative Concept: Separate Variables vs List

Method 1: Separate Variables

litres1 = 10

fat1 = 3.5

snf1 = 8.5

litres2 = 8.5

fat2 = 4.2

snf2 = 8.7

For 300 farmers, separate variables would be required for every entry. This approach becomes difficult to maintain and process.

Method 2: List of Entries

```
entries = [
    (10, 3.5, 8.5),
    (8.5, 4.2, 8.7),
    (12, 3.1, 8.3)
]
```

The program can process the entries using a loop:

```
for litres, fat, snf in entries:
    print(litres, fat, snf)
```

Complexity Comparison

Factor	**Separate Variables**	**List of Entries**
Processing time	O(n), if manually processed	O(n)
Storage	O(n) values, but difficult to manage	O(n)
Adding new farmers	Requires more variable definitions	Append a new entry

Maintenance	Difficult	Easy
Scalability	Poor	Good

Conclusion: A list of records is better because it organizes data systematically and supports 300, 6,000 or 150,000 farmers without changing the overall processing logic.

TASK 2 – Explore Values and Types in the Interpreter

Marks: 4 | Bloom's Level: K4 – Analyse

2.1 Python Data Types

Execute the following commands in Python interactive mode.

```
>>> type(10)
<class 'int'>

>>> type(10.0)
<class 'float'>

>>> type("10")
<class 'str'>

>>> type(True)
<class 'bool'>

>>> type([10, 8.5])
<class 'list'>
```

Explanation

- 10 is an integer.

- 10.0 is a floating-point number.
- "10" is a string because it is enclosed in quotation marks.
- True is a Boolean value.
- [10, 8.5] is a list containing two numeric values.

2.2 Division Operators

```
>>> 7 / 2
3.5

>>> 7 // 2
3

>>> 7 % 2
1

>>> -7 // 2
-4
```

Explanation:

- / performs ordinary division and returns a floating-point result.
- // performs floor division.
- % returns the remainder.
- -7 // 2 returns -4 because floor division rounds down towards negative infinity.

2.3 Floating-Point Precision

```
>>> 0.1 + 0.2 == 0.3
False
```

This happens because decimal fractions such as 0.1 and 0.2 cannot be represented exactly in binary floating-point format.

The stored result of 0.1 + 0.2 is slightly different from the stored value of 0.3.

Therefore, direct equality comparison produces False.

2.4 Why Float Is Not Recommended for Money

Floating-point calculations can produce small errors.

For example:

>>> 0.1 + 0.2

0.30000000000000004

Even small errors can accumulate when processing thousands of farmer payments.

Three alternatives are:

1. **Integer paise:** ₹30.50 is represented as 3050 paise.
2. **round():** Rounds a number to a specified number of decimal places, but does not eliminate all floating-point errors.
3. **Decimal:** Supports decimal arithmetic and is suitable for financial calculations.

For this assignment, Decimal is preferred.

2.5 Complete the Code – Decimal Total

```
from decimal import Decimal


payments = ["33.40"] * 1000 + ["0.10"] * 3



def total_float(items):
    total = 0.0
    for amount in items:
```

```
        total += float(amount)
    return total


def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total


print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))

print(
    0.1 + 0.2 == 0.3,
    Decimal("0.1") + Decimal("0.2") == Decimal("0.3")
)
```

Expected Output

```
float  : 33400.30000000061
Decimal: 33400.30
False True
```

```python
def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total


print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))

print(
    0.1 + 0.2 == 0.3,
    Decimal("0.1") + Decimal("0.2") == Decimal("0.3")
)
```

```
float  : 33400.30000000061
Decimal: 33400.30
False True
```

Explanation

The floating-point result contains a small representation error.

The Decimal result is exactly ₹33,400.30 because the decimal values are created directly from strings.

What Happens With Decimal(float(amount))?

Replace:

Decimal(amount)

with:

Decimal(float(amount))

The conversion first creates a binary floating-point approximation and then converts that approximation to a Decimal value.

Consequently, small representation errors are preserved, and the result may contain many decimal digits instead of exactly 33400.30.

Important: Use Decimal("33.40") rather than Decimal(33.40) for predictable decimal arithmetic.

2.6 Alternative Concept and Complexity Analysis

Integer Paise

```
payments_paise = [3340] * 1000 + [10] * 3


total_paise = sum(payments_paise)


print("Total:", total_paise / 100)
```

Output:

Total: 33400.3

The total is exact in integer paise. For final monetary formatting, convert the integer total to a Decimal or format the integer rupees and paise separately.

Float With Rounding

```
total = 0.0


for amount in payments:
    total = round(total + float(amount), 2)


print(total)
```

This example rounds after every addition. However, it is not a general guarantee of exact financial arithmetic because floating-point representation and rounding behaviour can still cause issues.

Complexity

Method	**Time**	**Storage**	**Exact monetary result?**
Float	O(n)	O(1) extra space	Not guaranteed
Decimal	O(n)	O(1) extra space	Yes, for exact decimal inputs and suitable precision
Integer paise	O(n)	O(1) extra space	Yes, when all amounts are represented in whole paise

To inspect the memory used by individual numeric values:

```
import sys

from decimal import Decimal


print(sys.getsizeof(3340))

print(sys.getsizeof(3340.0))

print(sys.getsizeof(Decimal("33.40")))
```

The exact sizes depend on the Python implementation and version. These values measure the individual objects, not the complete memory consumed by a list or program.